

AETHER - Agentic Security Orchestration & Automated Reasoning

By: Ritvik Indupuri

Date: 3/8/2026

Table of Contents

1. Executive Summary	3
2. System Architecture Deep-Dive	4
3. Component Specifications	5
4. MCP Integration & Protocol Architecture	7
5. Core Security Heuristics & Playbooks	9
6. Security, Privacy & Data Integrity	10
7. Conclusion	11

1. Executive Summary

AETHER is an elite agentic security orchestration platform designed to eliminate the latency between detection and remediation. By fusing the **Gemini 3 Pro** cognitive model with **CrowdStrike Falcon's** global telemetry, AETHER provides a semi-autonomous reasoning layer that acts as a force multiplier for Security Operations Centers (SOCs).

The platform transitions from the traditional "dashboard-only" approach to an "active reasoning" model. Instead of simply presenting alerts, AETHER interrogates the environment, correlates disparate telemetry points, and offers high-confidence tactical recommendations. This reduces the cognitive burden on analysts and ensures that critical decisions are supported by real-time data synthesis.

2. System Architecture

To maintain the high performance required for real-time security operations, AETHER is built on a modular, three-tier architecture. This design ensures that cognitive processing does not bottleneck telemetry ingestion, creating a seamless pipeline from raw signal to executive conclusion.

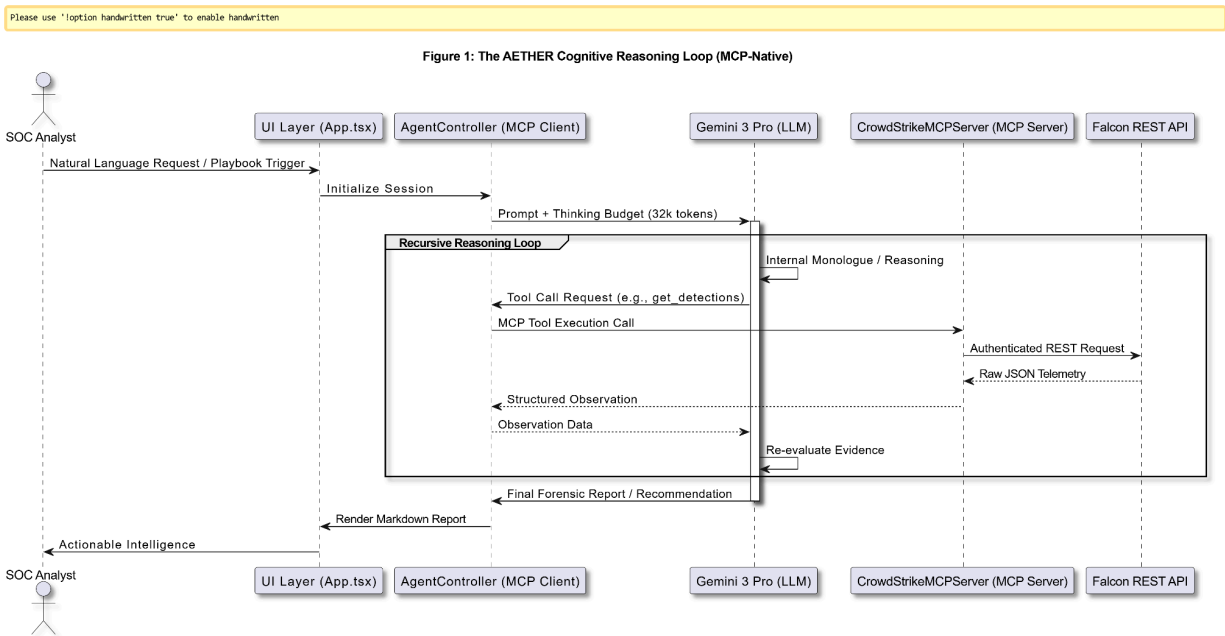


Figure 1: AETHER System Architecture Diagram

Operational Analysis:

1. **Analyst Interaction:** The cycle initiates at the UI layer, where user intent is captured via natural language or playbook triggers and passed to the orchestration core.
2. **Cognitive Core:** The `AgentController` acts as an MCP client, initializing a Gemini 3 Pro session. The model uses a 32,768-token "Thinking Budget" to parse the request and identify telemetry gaps.
3. **MCP Infrastructure:** When the model identifies a need for data, it issues a `functionCall` which the `AgentController` routes through the appropriate MCP Server.
4. **Tactical Execution:** The `CrowdStrikeMCPServer` executes the request via the CrowdStrike REST API.
5. **Recursive Observation:** API results are returned through the MCP layer to the model as "Observations." The model re-evaluates its reasoning, potentially issuing further tool calls or synthesizing a final forensic report if the evidence is sufficient.

3. Component Specifications

The transitions between reasoning and execution are facilitated by five primary software components, each optimized for specialized roles within the security lifecycle.

3.1 AgentController (The Orchestration Core)

The `AgentController` serves as the primary interface between the user interface and the cognitive reasoning engine. It is responsible for managing complex chat histories and maintaining the recursive logic required for multi-step forensic investigations. Functioning as a Model Context Protocol (MCP) client, it dynamically discovers tools from registered MCP servers and utilizes a specific 32,768-token thinking budget to allow the model to pre-visualize process trees and network flows before generating a final response. This core component implements an asynchronous loop that automatically resolves tool requests, allowing the AI to execute a sequence of multiple API calls—such as listing incidents, fetching host details, and retrieving detections—to find a root cause without requiring further manual input from the analyst.

3.2 CrowdStrikeMCPServer (The Tool Provider)

This specialized implementation of the MCP MCPServer interface acts as the secure bridge that exposes CrowdStrike Falcon capabilities to the AI reasoning core. By implementing the standardized `listTools` function, the server provides the AI with a consistent schema of available security operations, including critical tools like host containment and detection retrieval. It further manages execution routing through the `callTool` method, which maps AI-generated function calls directly to the underlying service logic. This standardized contract ensures that the reasoning engine can reliably interact with the Falcon environment regardless of the underlying API's inherent complexity.

3.3 CrowdStrikeService (The Tactical Gateway)

The `CrowdStrikeService` is a robust, class-based integration layer designed for the Falcon REST API, focusing on high-fidelity telemetry retrieval and kernel-level host actions. It features a self-healing OAuth2 authentication flow that proactively monitors token expiration, refreshing credentials 60 seconds before they expire to prevent session drops during critical investigations. To ensure architectural resilience, the service includes specialized error handling for a variety of API states, such as decommissioned endpoints or permission denials, providing the user interface with clear and actionable feedback if scopes are insufficient for a requested tactical action.

3.4 App.tsx (Global State & Autonomous Monitoring)

Serving as the application's central nervous system, the root App component maintains the global state and monitors the overall health of the security environment. It incorporates an Auto-Pilot Engine, consisting of a 30-second polling loop that scans for critical incidents and automatically initiates investigations by injecting system alerts into the chat stream. This enables autonomous triage even when an analyst is away from the console. Additionally, the component functions as an API State Tracker that monitors whether the platform is in an idle, stable, or error state, which in turn dictates UI interactivity and manages the command interface logic.

3.5 ChatMessage.tsx (Forensic Rendering)

The `ChatMessage` component is responsible for transforming raw telemetry and complex AI reasoning into a clinical, readable forensic timeline for the SOC analyst. It provides comprehensive Markdown support for rendering analytical reports containing technical tables, code blocks, and structured lists. A key feature of this component is the "Show Your Work" tactical modules, which display the raw inputs and outputs of every tool call. This transparency allows analysts to verify the specific data sources used by the AI, maintaining the integrity of the forensic investigation and ensuring full human oversight of all automated reasoning cycles.

4. MCP Integration & Protocol Architecture

AETHER leverages the **Model Context Protocol (MCP)** to decouple the reasoning engine from the tactical execution layer. This standardization allows for seamless tool discovery and robust error handling across disparate security services.

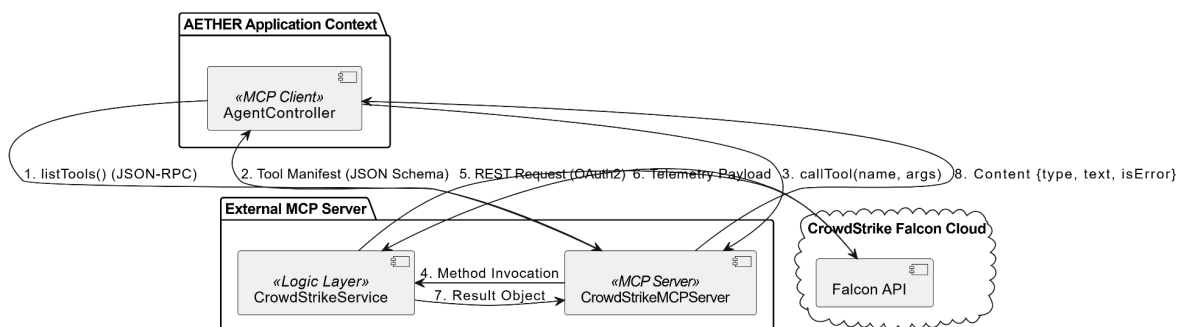


Figure 1: AETHER MCP Architecture Diagram

Protocol Implementation Details:

The interaction between the MCP Client (AETHER) and the MCP Server (CrowdStrike) follows a strict request-response lifecycle defined by the JSON-RPC 2.0 standard.

1. **4.1 The MCP Client (AgentController):** During initialization, it calls `listTools()`. These tools are converted into Gemini-compatible `FunctionDeclarations`.
 - **Schema Translation:** The client automatically maps MCP JSON Schemas to Gemini's Type system (e.g., mapping integer to `Type.INTEGER`).
 - **Context Injection:** Server names are injected into system instructions, e.g.,
`[crowdstrike-falcon-mcp] get_detections.`
2. **4.2 The MCP Server (CrowdStrikeMCPServer):** Implements `listTools()` and `callTool()`, providing a consistent contract regardless of API complexity.
 - **Error Normalization:** Tool execution errors are caught and returned as structured `isError: true` responses, preventing reasoning loop destabilization.
3. **4.3 Tool Execution Lifecycle:**
 - **Discovery:** Agent fetches schemas upon boot.
 - **Inference:** Gemini identifies a tactical need and generates a `functionCall`.
 - **Dispatch:** Agent routes the call to the specific MCP Server.
 - **Translation:** Server maps the tool name to a `CrowdStrikeService` method.
 - **Observation:** The result is returned and incorporated into the next "Thinking" cycle.

5. Core Security Heuristics & Playbooks

AETHER transitions from raw data to intelligence by utilizing pre-defined "Playbooks"—structured heuristics that guide the AI's reasoning toward specific investigative outcomes.

5.1 Root Cause Analysis (RCA)

This playbook instructs the AI to pull a detection and "Walk the Tree." It retrieves parent process IDs (PPIDs) recursively to find the original execution point.

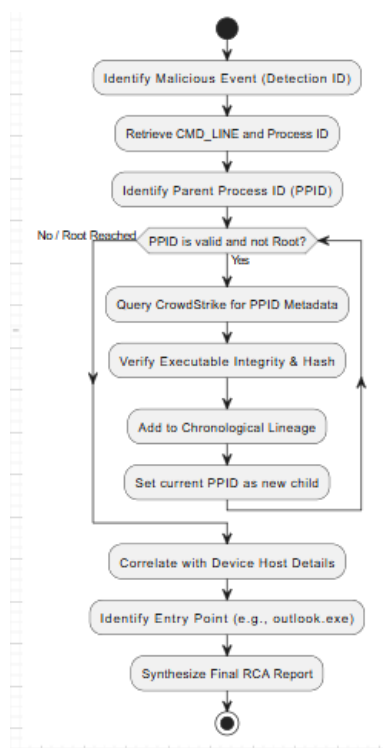


Figure 1: Root Cause Analysis Workflow

Logic Synthesis:

The AI first uses `get_detections` to find the `cmd_line` of a malicious event. It then correlates this with `get_device_details` to verify host integrity. By comparing timestamps and process IDs, it constructs a chronological attack lineage identifying entry points (e.g., `outlook.exe -> powershell.exe -> mimikatz.exe`).

5.2 MITRE ATT&CK Framework Mapping

The system instruction enforces strict adherence to the MITRE ATT&CK framework to ensure all findings are categorized within industry-standard adversarial tactics and techniques. Telemetry data ingested from the Falcon `/detects` endpoint frequently contains specific technique identifiers, such as T1059.001 for PowerShell abuse or T1003 for OS Credential Dumping. AETHER parses these fields in real-time and applies high-visibility CSS styles to the resulting forensic report, allowing analysts to immediately align the current incident with known adversary behaviors. This automated mapping facilitates more effective reporting and ensures that tactical remediation steps are directly informed by the specific stage of the attack lifecycle identified by the model.

6. Security, Privacy & Data Integrity

The AETHER platform is architected around strict "Zero-Trust" principles at the application layer to maintain the highest standards of data integrity and privacy. All sensitive information, including API keys and client secrets, is processed exclusively within the local browser context of the client, ensuring that telemetry data and authentication credentials are never transmitted to unauthorized intermediary servers outside of the official Google and CrowdStrike cloud environments. This localized execution model minimizes the attack surface and places full control of the data lifecycle in the hands of the SOC analyst.

Furthermore, the platform adheres to the Principle of Least Privilege by requiring a specific set of granular API scopes to execute its forensic functions. Access is restricted to Read-only permissions for Alerts, Detections, and Incidents, while Host management requires both Read and Write scopes to facilitate necessary kernel-level containment actions during active threats. To prevent broken or unauthenticated investigation states, the system performs a mandatory handshake verification upon session entry. This process validates the Client Secret and connection stability before allowing any investigative commands to be issued, thereby guaranteeing that all reasoning cycles are supported by a secure and verified telemetry stream.

7. Conclusion

AETHER transforms the traditional SOC workflow from manual data-gathering to high-level strategic oversight. By delegating the clinical labor of telemetry correlation to an agentic reasoning core, analysts can spend their time on what matters most: stopping the adversary.

As security landscapes become more complex, AETHER's modular architecture allows it to evolve, incorporating new tools and higher-fidelity reasoning models as they become available, ensuring a future-proof defense posture.